За пределами сжатия контекста: Состав состояния и недавняя история программирующего агента

Alex Markov-Buturskiy

Аннотация

Структурированное состояние обещает ограничить контекст агента, заменив растущую историю взаимодействия компактной памятью. Однако размер промпта не определяет, сколько работы потребуется для решения задачи. Мы исследуем, какие сведения должно сохранять состояние программирующего агента и кто должен их записывать. Сравниваются память, поддерживаемая моделью, гибридное состояние с автоматическими записями среды исполнения и сохранение недавней истории; отдельно проверяются схема и протокол обновления. При замене истории состоянием модели пиковый контекст на общих успехах сокращается до 0.47 от контрольного, но стоимость возрастает в 2.57 раза, а общая успешность снижается с 61.3% до 34.9%. Поисковое сравнение состава гибридной памяти показывает преимущество записей изменений и команд перед одним списком файлов, без устойчивого улучшения от каждого следующего компонента. В повторных измерениях сохранение недавней истории повышает успешность состояния модели до 59.0%, гибридной памяти — до 48.3% при 51.7% у её собственного контроля. Проверенные изменения схемы и inline-протокол такого восстановления не дают. Стоимость на успех при сохранении истории снижается в первой реализации, но превышает контрольную во второй. Результаты показывают пользу компактной памяти вместе с границами её применения: содержание автоматических записей и доступ к исходным взаимодействиям требуют отдельных решений, а экономию контекста следует оценивать вместе с качеством и стоимостью выполнения.

1 Введение

Программирующий агент узнаёт задачу через действия. Команда обнаруживает зависимость, тест опровергает исправление, сообщение компилятора связывает ошибку с конкретной правкой. Последующие решения зависят от того, какие сведения об этих действиях остались доступны. Если история взаимодействия растёт на каждом шаге, вместе с ней растёт и объём входа модели. Если её сжать, приходится решать, чем заменить удалённые наблюдения.

SKILL.state предлагает хранить изменяемое структурированное состояние вместо полного транскрипта [Badhe et al., 2026]. Привлекательность этого решения состоит в отделении текущей рабочей памяти от накопленной истории. При ограниченном размере состояния контекст не обязан расти с каждым действием. Но ограничение размера представления само по себе не ограничивает объём информации, необходимой для решения задачи. Краткая запись о неудаче может не сохранить сообщение об ошибке; список изменённых файлов не сообщает, что именно было изменено и почему.

Из этого возникает инженерный вопрос: какую работу по ведению памяти следует поручить модели? Гипотезы и планы требуют интерпретации. Выполненная команда и её наблюдённый результат уже известны среде исполнения. Повторная генерация этих сведений моделью расходует выходные токены и делает их сохранность зависимой от обновлений семантического состояния. Разделение обязанностей позволяет хранить наблюдения автоматически, оставляя модели рассуждение об их значении. Мы называем такое представление гибридным состоянием.

В работе проверяются возможности этого разделения на задачах программирующего агента. Сначала оценивается цена замены истории состоянием модели. Затем сравнивается состав автоматической памяти: от ссылок на файлы до записей изменений, команд, диагностик и результатов тестов. Следующие сравнения проверяют, устраняют ли оставшееся отставание недавняя история, адаптация схемы или другой протокол обновления. Качество рассматривается вместе с ресурсами, поскольку меньший промпт может сочетаться с большей генерацией и более высокой стоимостью успешного выполнения.

Результаты дают различающиеся ответы на эти вопросы. Компактное состояние действительно уменьшает пиковый контекст, но без истории теряет успешность. В поисковом сравнении более содержательные записи среды исполнения заметно лучше бедного списка файлов; дальнейшее обогащение не даёт последовательного улучшения. В повторных измерениях наиболее выраженное восстановление успешности связано с недавней историей. Испытанные схема и inline-протокол не обеспечивают аналогичного результата. При этом экономический эффект сохранения истории различается между реализациями.

Вклад работы состоит в эмпирическом разделении решений, которые легко объединить под названием «память агента»: кто записывает сведения, что входит в состояние, какие взаимодействия остаются в исходном виде и как оплачивается дальнейшая работа. Это исследование переноса идеи SKILL.state в другой агентный стек, а не повторение исходных экспериментов. Его результаты характеризуют испытанные реализации и уточняют, какие изменения частично компенсируют их ограничения.

2 Подходы к памяти и исследовательские вопросы

История и состояние по-разному сохраняют информацию. История оставляет записи взаимодействия в исходной последовательности. Состояние преобразует выбранные сведения в описание текущей задачи. Поле состояния может сообщать, что тест завершился неудачно, но не сохранять полный вывод теста и предшествующие правки. В то же время наличие исходной последовательности не гарантирует, что модель правильно ею воспользуется.

Различается и способ формирования состояния. Модель может поддерживать гипотезы, план и описание незавершённой работы. Среда исполнения может записывать непосредственно наблюдаемые события: выполненные команды и их результаты. Представление, объединяющее эти источники, далее называется *гибридным состоянием*. Автоматическая запись уменьшает необходимость заново описывать события, но сама по себе не даёт ни их интерпретации, ни плана следующего действия.

Опубликованный контракт SKILL.state объединяет изменение состояния и действие в одном ответе модели [Badhe et al., 2026]. Отдельный инструмент обновления меняет этот порядок: ведение памяти становится инструментальной операцией внутри цикла агента. Вариант с inline-обновлением проверяет совместную передачу изменения и действия, а сравнение схем — организацию записываемого содержания. Эти изменения относятся к реализации переноса, но не устраняют все отличия от опубликованной системы.

Исследование начинается с сопоставления сжатия контекста и стоимости решения. Далее проверяются состав автоматических записей, сохранение недавней истории и изменения схемы с протоколом. Эти сравнения различают содержимое памяти, источник её обновлений и доступность исходных взаимодействий. Для ответа нужны сопоставления с собственным контролем каждого подхода. В частности, добавление истории к состоянию проверяет пользу этой истории внутри системы со состоянием, но не позволяет выделить пользу самого блока состояния.

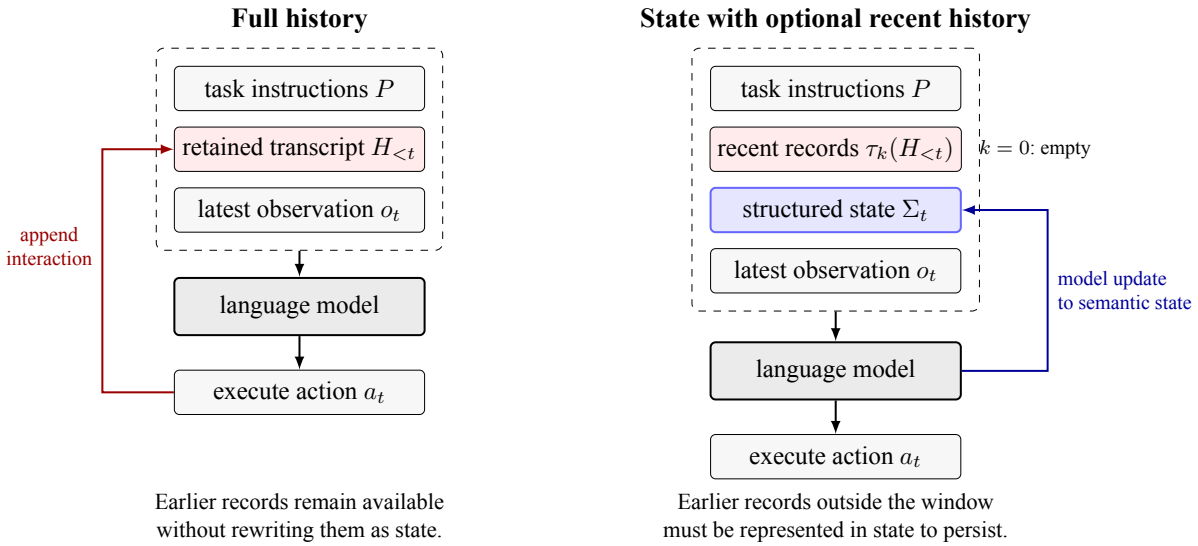


Figure 1: Context policies. State replaces the older transcript, while the retention parameter k controls the recent suffix. The arrows show information flow, not the number of model calls.

3 Контекст агента и протоколы обновления

Обозначим инструкции задачи через P, текущее наблюдение через o_t, а предшествующие сообщения и результаты инструментов через $H_{<t}$. При сохранении полной истории

$$C_t^{\text{full}} = [P, H_{<t}, o_t]. \tag{1}$$

Структурированная память формирует контекст

$$C_t^{\text{state},k} = [P, \Sigma_t, \tau_k(H_{<t}), o_t], \qquad \tau_0(H) = \varnothing, \tag{2}$$

где τ_k выбирает доступные недавние записи, а $\tau_\infty(H) = H$. При $k = \infty$ сохраняются *и* состояние, *и* весь транскрипт; контроль с полной историей без состояния — другая политика (рис. 1). Доступ к скрытым рассуждениям не предполагается.

3.1 Кто записывает состояние и как оно обновляется

В первом подходе модель поддерживает семантическое состояние S_t. Гибридная память добавляет записи среды исполнения V_t:

$$\Sigma_t = (S_t, V_t), \qquad S_{t+1} = \text{Apply}(S_t, \Delta S_t), \qquad V_{t+1} = \text{Record}(V_t, a_t, r_t). \tag{3}$$

Модель ведёт гипотезы и незавершённую работу; рантайм сохраняет наблюдаемые события инструментов r_t: от ссылок на файлы до изменений, результатов команд, диагностик, тестов и неудачных попыток. Обновление S не может изменить V, но полного захвата событий это не гарантирует. Недавняя история может дополнять оба типа памяти.

Отдельный инструмент принимает изменения состояния; inline-протокол объединяет обновление и действие в ответе $\{\Delta S_t, a_t\}$, следуя контракту SKILL.state [Badhe et al., 2026]. Алгоритмы 1 и 2 — опорные контракты, а не восстановленные трассы.

Algorithm 1 Context selection and state ownership

Require: Instructions P, policy π, retention setting k, update protocol q, initial observation o

Initialize history H, semantic state S, runtime record V
while the task has not terminated **do**
    if $\pi =$ FullHistory **then**
        $C \leftarrow [P, H, o]$
    else
        $\Sigma \leftarrow S$ **or** (S, V) according to state ownership
        $C \leftarrow [P, \Sigma, \tau_k(H), o]$
    end if
    $(S', a) \leftarrow \text{PolicyStep}(C, S, \pi, q)$
    $r \leftarrow \text{Execute}(a)$; $S \leftarrow S'$
    if hybrid memory is enabled **then**
        $V \leftarrow \text{Record}(V, a, r)$
    end if
    Append a record of (o, a) to H; $o \leftarrow r$
end while

Algorithm 2 Inline patch–action contract

Require: Current context C and semantic state S

$(\Delta S, a) \leftarrow \text{Model}(C)$
if $\text{Valid}(\Delta S, S)$ **then**
    return $(\text{Apply}(S, \Delta S), a)$
else
    Keep S unchanged and return a validation observation
    Do not execute a from the rejected envelope
end if

3.2 Границы контекста и стоимость задачи

Полная история даёт $O(T^2)$ накопленного входа за T действий; ограниченное состояние и постоянное окно дают $O(T)$. Ни одна оценка не фиксирует само T. При накопленных некешированном входе U, чтении из кеша K, выходе O и тарифах α, γ, β

$$D = \alpha U + \gamma K + \beta O. \tag{4}$$

Ведение состояния может увеличить выход, снизить переиспользование кеша или изменить T, поэтому меньший пиковый промпт не определяет D.

4 Методика эксперимента

4.1 Агент и сравнения

Оценивается `zai/glm-5.3-flash` с Pi 0.85.1 и Harbor 0.22 на Terminal-Bench 2.0 [Merrill et al., 2026], лимит — 200 ходов. Успех — двоичный вердикт проверяющей программы задачи. Анализируемое подмножество содержит 87 задач; две задачи, чьи окружения не удалось инициализировать, исключены.

Приводятся четыре сравнения, каждое со своим контролем полной истории, сопоставленным по задаче и повтору; контроли не объединяются.

- *Состояние модели*: четыре условия, три повтора, $k \in \{0, 2, 6\}$.

- *Гибридное состояние*: восемь условий, два повтора. Варианты убирают записи неудач, добавляют недавнюю историю или сочетают способ обновления (через инструмент или в тексте ответа) со схемой (исходной или адаптированной).
- *Состав автоматических записей*: четыре набора и контроль, один повтор. Контроль и самые богатые наборы пересекаются с первым гибридным повтором, поэтому это не независимое воспроизведение.
- *Пилот окон*: 12 задач, шесть условий ($k \in \{0, 2, 6, 12, \infty\}$ и контроль), три повтора. Сохранились только итоговые сводки.

Схема адаптировалась на тех же задачах, а слабые наборы записей отброшены после предварительных прогонов, поэтому сравнения поисковые и внутривыборочные. Во всех реализациях k — заданная настройка удержания; равное k не гарантирует одинаковой сериализации.

4.2 Единый набор показателей

Каждое сравнение приводится в двух панелях с одинаковыми строками. *Панель качества*: доля решённых задач по всем включённым исходам, медиана ходов T, медианная нормированная стоимость, медиана парного отношения стоимости к полной истории и решения на доллар $\sum Y / \sum D$ (с учётом неудач). *Панель ресурсов*: медианы пикового промпта, накопленного некешированного входа U, чтения из кеша K, выхода O и cache hit

$$h_{iar} = \frac{K_{iar}}{U_{iar} + K_{iar}}, \tag{5}$$

который усредняется медианой по выполнениям, а не делением медиан токенов. Счётчика записи в кеш нет. Нормированная стоимость

$$D = 10^{-6}(0.15U + 0.03K + 0.50O) \tag{6}$$

выражена в долларах США и не является счётом провайдера. Показатели, которые сохранившиеся записи не позволяют восстановить (пики и cache hit гибридов, доли решённых и входные счётчики пилота), обозначены – и не подставляются.

4.3 Статистический анализ и когорты

Сопоставленные исходы проверяются двусторонним точным тестом Мак-Немара; для состояния модели добавлены 95% бутстрэп-интервалы по задачам (100 000 выборок, seed 20260922) с сохранением всех условий и повторов задачи. Проверки поисковые, без поправки на множественность; незначимость не означает эквивалентности. Когорты по горизонту задаются медианой ходов контроля: короткая $T \leq 8$ (31 задача), средняя $8 < T \leq 20$ (29), длинная $T > 20$ (27); состав фиксирован для всех условий и описывает поведение контроля, а не назначенную сложность. Повторяемость в пилоте — число вызовов инструментов, делённое на число различных пар инструмент/аргументы; 1.00 означает отсутствие идентичных повторов, а не отсутствие смысловой избыточности. Доступные материалы перечислены в дополнении S1.

5 Сжатие контекста не равно экономии на задаче

Состояние модели сокращает контекст, но не сохраняет результат автоматически. На общих успехах медианное отношение пикового промпта к контролю равно 0.47, стоимости — 2.57, выходных токенов — 4.55. По всем включённым исходам успешность снижается с 61.3% до 34.9%; бутстрэп-интервал разности равен $[-34.1, -18.8]$ процентного пункта (дополнение S2).

Cache hit обнаруживает ещё одну цену замены истории. Медианная доля входа, обслуженного из кеша, составляет 83.3% для полной истории, 49.4% для состояния без истории, 37.4% при $k = 2$

и 20.3% при $k = 6$. Растущий транскрипт может повторно использовать неизменный префикс; обновляемое состояние и движущееся окно могут сокращать такое использование. Измерения подтверждают различие поведения кеша, но не выделяют конкретную деталь сериализации как его причину.

Сам по себе низкий cache hit не делает политику невыгодной. Окно с наименьшим попаданием в кеш одновременно даёт наибольшую успешность среди проверенных вариантов состояния. Полная цена складывается из кешированного и некешированного входа, генерации и числа действий. Для состояния без истории стоимость на наблюдаемый успех равна \$0.1630 против \$0.1116 у контроля. В дальнейших сравнениях сохраняются одинаковые колонки качества, денег, токенов и кеша: изменение представления памяти нужно оценивать по всем этим измерениям.

6 Что должна запоминать среда исполнения?

Гибридное состояние позволяет рантайму хранить наблюдаемые результаты команд, а модели — гипотезы и незавершённую работу (рис. 2). Польза зависит от содержания записей: знать, какой файл просматривался, — не то же, что знать, что изменилось и какие команды последовали.

6.1 От ссылок на файлы к свидетельствам выполнения

Одни ссылки на файлы дают 28.7% решённых задач (табл. 1). Добавление изменений и команд поднимает долю до 40.2%, снижает медианный выход с 23.2 до 16.5 тысячи токенов, а медианную стоимость — с \$0.01406 до \$0.01016. Диагностика, среда и тесты дают 42.5%, записи неудачных попыток — 41.4%. Ни один вариант не достигает полной истории (55.2%). Выигрыш находится между ссылками на файлы и записями выполнения; дальше наступает плато (рис. 3). Строки меняют группы записей, поэтому отдельный компонент не изолирован.

У всех вариантов автоматических записей cache hit ниже, чем у полной истории (42.1–54.6% против 82.7%): это согласуется с тем, что меняющийся блок состояния ломает переиспользование префикса, которое растущий транскрипт сохраняет. Решения на доллар растут с 12.87 до 20.79, приближаясь к контрольному 22.39, но не превышая его. На задачах, решённых всеми вариантами, богатейшие наборы стоят 1.03 и 1.17 от полной истории (дополнение S4): богатые записи лучше бедного индекса, но преимущества в цене над транскриптом нет.

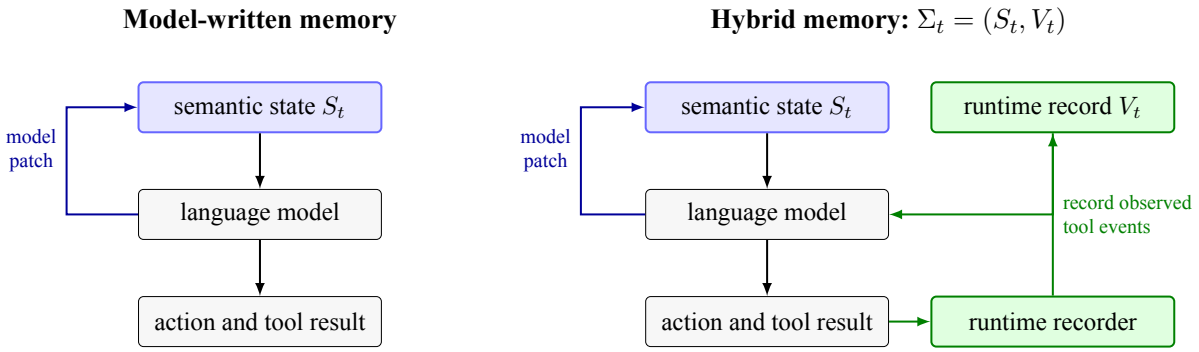


Figure 2: State ownership. Hybrid memory retains model-written semantic content and adds runtime-derived records. Recording an event does not establish that an extracted interpretation is correct or that all relevant events were captured.

Table 1: Состав автоматических записей при одной семантической схеме (поисковое сравнение, один повтор). Сверху — панель качества по всем включённым исходам, снизу — панель ресурсов (медианы по выполнениям). Строки с + добавляют записи к предыдущей.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	55.2	11	0.01139	1.00	22.39
File references	28.7	15	0.01406	1.03	12.87
Files, changes, commands	40.2	9	0.01016	0.98	20.79
+ diagnostics, environment, tests	42.5	9	0.01316	1.01	20.92
+ failed-attempt records	41.4	9	0.01196	1.01	21.19

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	13.4	20.4	61.2	16.4	82.7
File references	3.9	15.9	17.9	23.2	54.6
Files, changes, commands	5.3	12.8	14.7	16.5	42.1
+ diagnostics, environment, tests	5.8	16.8	16.4	19.1	42.4
+ failed-attempt records	5.6	13.3	15.6	18.6	43.2

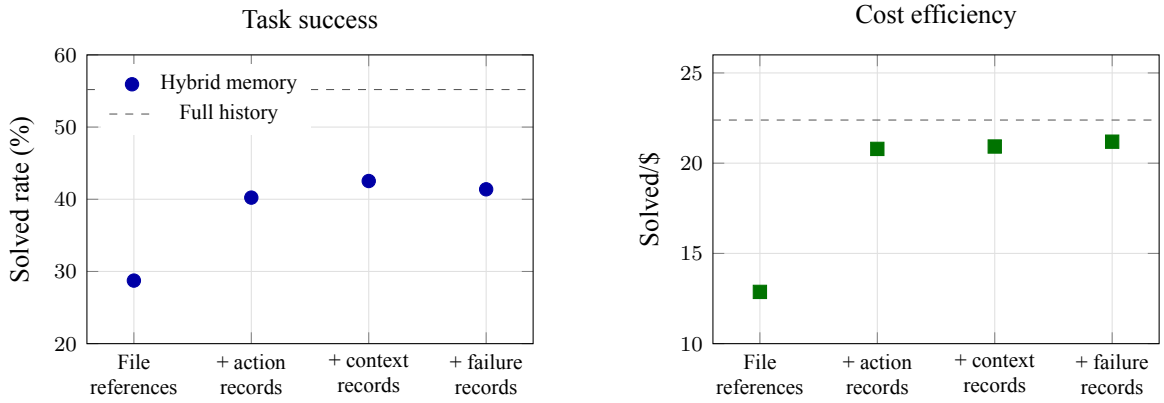


Figure 3: Exploratory comparison of cumulative runtime-owned information. Each position adds records to the preceding configuration: action records cover file changes and commands; context records cover diagnostics, environment, and tests. Efficiency includes the cost of unsuccessful episodes. Dashed lines show the same full-history reference. These descriptive observations share their richer configurations and reference with the first repeat of the final comparison; they do not isolate individual components or establish a monotonic benefit.

6.2 Закрывают ли записи неудач оставшийся разрыв?

В итоговом гибридном сравнении записи неудач меняют успешность с 38.5% до 39.7% при 51.7% у полной истории (табл. 2); парная разница мала. Сохранённый результат не обязательно передаёт порядок и контекст, нужные позднему решению. Новые поля разрыв не закрыли; остаётся кандидат — сохранить часть исходных взаимодействий, что проверяется далее.

Table 2: Итоговое гибридное сравнение с записями неудач и без них. Те же две панели, что и в табл. 1. Гибридные сводки не позволяют восстановить парные отношения стоимости, пиковый промпт и cache hit (–).

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	51.7	11	0.0104	–	21.13
Hybrid, no failure records	38.5	8	0.0127	–	20.12
Hybrid state	39.7	8	0.0120	–	21.36

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	–	20.3	62.5	16.4	–
Hybrid, no failure records	–	16.6	10.2	19.1	–
Hybrid state	–	14.5	9.3	17.3	–

7 Сколько недавней истории достаточно?

7.1 Восстановление успешности и его цена

Состояние удерживает выбранные сведения; недавняя история сохраняет также формулировки, порядок и контекст взаимодействий. Её добавление повышает успешность состояния модели с 34.9% при $k = 0$ до 45.2% при $k = 2$ и 59.0% при $k = 6$. Для последнего варианта разность с полной историей равна -2.3 процентного пункта при интервале $[-7.7, +3.4]$. Это существенное восстановление, но не доказанная эквивалентность.

Гибридная память показывает то же направление: 39.7% без истории, 40.8% при $k = 2$ и 48.3% при $k = 6$. Последний вариант даёт $+8.6$ процентного пункта относительно гибридного состояния без истории ($p = 0.011$); собственный контроль достигает 51.7%. При этом стоимость на наблюдаемый успех возрастает с \$0.0468 до \$0.0501. Для состояния модели она, напротив, снижается с \$0.1630 до \$0.0768. Одинаковое решение о сохранении истории имеет разные экономические последствия.

7.2 Полная последовательность окон: почему шесть?

Одно восстановление при $k = 6$ не показывает, что более широкое окно ничего не добавляет. Поэтому пилот включал также $k = 12$ и $k = \infty$ (состояние плюс весь транскрипт; это не контроль без состояния) с тремя повторами на 12 задачах (табл. 5 и 6).

Три показателя перестают меняться при $k = 6$. Число решённых задач растёт с 9 из 12 при $k \leq 2$ до 12 при $k = 6$ и дальше не меняется; отношение повторов падает с 1.27 до 1.00 и остаётся там; строки $k = 6$, 12 и ∞ отличаются от контроля одной несовпадающей задачей. Две из этих линий являются границами, а не плато. Решение всех 12 задач задаёт потолок набора, а отношение, равное единице, является нижней границей отношения повторов. Пилоту из 12 задач нужно

Table 3: Удержание недавней истории: панель качества. Учитываются все включённые исходы; у каждого семейства памяти свой контроль полной истории. Отношение стоимости — медиана парных отношений, а не отношение медиан; недоступные гибридные отношения оставлены пустыми. Solved/$ использует суммарную стоимость с учётом неудач.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Model-written memory					
Full history	61.3	13	0.0137	1.00	8.96
Model-written state	34.9	25	0.0239	1.59	6.14
State + history ($k = 2$)	45.2	14	0.0151	1.14	9.16
State + history ($k = 6$)	59.0	11	0.0154	1.05	13.02
Hybrid memory					
Full history	51.7	11	0.0104	–	21.13
Hybrid state	39.7	8	0.0120	–	21.36
Hybrid + history ($k = 2$)	40.8	8	0.0122	–	21.65
Hybrid + history ($k = 6$)	48.3	9	0.0144	–	19.95

Table 4: Панель ресурсов для тех же политик истории. Пиковый промпт и токены — медианы по выполнениям; cache hit — медиана доли кеша внутри выполнения. Пики и cache hit гибридов недоступны, а не равны нулю.

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Model-written memory					
Full history	16.1	28.8	87.4	16.4	83.3
Model-written state	4.0	24.9	36.7	37.4	49.4
State + history ($k = 2$)	6.2	30.3	19.1	20.1	37.4
State + history ($k = 6$)	11.3	53.3	14.8	16.6	20.3
Hybrid memory					
Full history	–	20.3	62.5	16.4	–
Hybrid state	–	14.5	9.3	17.3	–
Hybrid + history ($k = 2$)	–	22.4	6.4	16.6	–
Hybrid + history ($k = 6$)	–	49.0	9.9	16.4	–

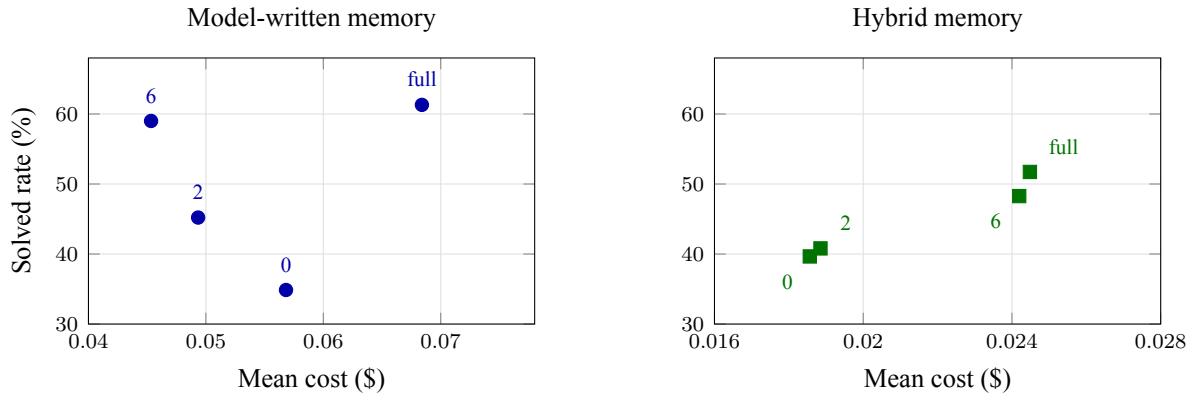


Figure 4: Success–cost trade-offs when retaining history. Labels are the retention setting k; full denotes the full-history reference without a state block. Each panel includes failures in mean cost and uses its own control. Hybrid mean costs use rounded reported totals. The horizontal scales differ.

Table 5: Пилот окон в стандартных двух панелях (окончательные медианы по трём повторам). Доли решённых по выполнениям, эффективность, объединённые отношения стоимости и входные счётчики не сохранились (–). Cache hit — зарегистрированное значение пилота, а не реконструкция по счётчикам.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	–	12	0.0035	–	–
Model-written state	–	25	0.0099	–	–
State + history ($k = 2$)	–	15	0.0054	–	–
State + history ($k = 6$)	–	12	0.0070	–	–
State + history ($k = 12$)	–	9	0.0059	–	–
State + history ($k = \infty$)	–	10	0.0060	–	–

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	9.87	–	–	6.54	86.5
Model-written state	3.46	–	–	29.62	64.3
State + history ($k = 2$)	6.18	–	–	14.20	48.8
State + history ($k = 6$)	8.43	–	–	12.16	27.1
State + history ($k = 12$)	11.34	–	–	7.93	27.0
State + history ($k = \infty$)	14.02	–	–	9.11	38.2

Table 6: Исходы пилота окон по журналу исследования (исходные сессии недоступны). Задача считается решённой, если прошёл хотя бы один из трёх повторов. Несовпадающие пары и точный p Мак-Немара — относительно полной истории. Выход на ход — зарегистрированная медиана.

Approach	Tasks solved, % (any repeat)	Discordant (gain/loss)	McNemar p	Median calls / distinct calls	Output tok per turn
Full history	91.7	–	–	1.00	528
Model-written state	75.0	0/2	0.500	1.27	1105
State + history ($k = 2$)	75.0	0/2	0.500	1.06	1116
State + history ($k = 6$)	100.0	1/0	1.000	1.00	1053
State + history ($k = 12$)	100.0	1/0	1.000	1.00	1059
State + history ($k = \infty$)	100.0	1/0	1.000	1.00	967

не меньше шести односторонних несовпадений для значимости на уровне пяти процентов, поэтому он не различает условия у потолка. Корректный вывод: порог лежит между двумя и шестью сообщениями, а не «более широкие окна бесполезны».

Ресурсные столбцы продолжают меняться. Медианный пик промпта растёт с 8.43 тысячи токенов при $k = 6$ до 11.34 и 14.02; медиана ходов снижается с 12 до 9 и 10; медианная стоимость — \$0.0070, \$0.0059 и \$0.0060. Парные отношения стоимости остаются в диапазоне 1.55–2.25 для всех $k \geq 6$ во всех повторах (дополнение S5). Причина видна по выходу на ход: каждое условие с состоянием пишет примерно вдвое больше контрольных 528 токенов (1053 при $k = 6$), поскольку модель переписывает состояние на каждом шаге. Расширение окна не убирает эти постоянные издержки, а лишь меняет меньшее число ходов на более длинные промпты. Cache hit также остаётся низким (27.1–38.2% против 86.5%), что согласуется с блоком состояния, который стоит перед транскриптом в формуле (2) и меняется каждый ход.

Итак, $k = 6$ — наименьшее проверенное окно, при котором успешность и избыточные повторные вызовы достигают предела разрешения пилота; самая дешёвая точка за ним этими данными не определена. Основное сравнение состояния модели подтверждает восстановление при $k = 6$, но $k = 12$ и $k = \infty$ в нём не запускались; различающее продолжение требует более трудных задач с запасом над потолком и контроля недавней истории без состояния.

8 Как длина траектории меняет компромисс

За совокупным результатом скрываются различия между задачами с короткими и длинными траекториями. Используем когорты из раздела 4: короткую ($T \leq 8$), среднюю ($8 < T \leq 20$) и длинную ($T > 20$), где T — медианная длина траектории полной истории для данной задачи. Состав каждой когорты одинаков для всех способов организации памяти. Собственная длина траектории метода является результатом, а не основанием для его отнесения к когорте.

Table 7: Когорты длины траектории, заданные контролем: качество и стоимость. Учитываются все включённые исходы. Отношение стоимости — медиана отношений к полной истории в той же задаче и том же повторе.

Approach	Solved rate (%)	Median T	Median cost (\$)	Cost ratio ($\times$)	Solved/\$
Short: control median $T \leq 8$					
Full history	47.3	5	0.0086	1.00	6.89
Model-written state	29.0	4	0.0089	1.01	12.36
State + history ($k = 2$)	36.6	6	0.0088	1.02	10.70
State + history ($k = 6$)	46.2	6	0.0097	1.06	10.55
Medium: control median $8 < T \leq 20$					
Full history	75.9	11	0.0089	1.00	33.54
Model-written state	55.2	35	0.0283	3.48	9.88
State + history ($k = 2$)	64.4	14	0.0113	1.59	17.99
State + history ($k = 6$)	73.6	10	0.0100	1.48	28.37
Long: control median $T > 20$					
Full history	61.7	36	0.0523	1.00	5.27
Model-written state	19.8	101	0.0852	1.30	2.05
State + history ($k = 2$)	34.6	47	0.0585	0.91	4.25
State + history ($k = 6$)	58.0	21	0.0414	0.66	8.55

Состояние без истории уступает контролю по успешности во всех когортах. Наибольший разрыв возникает в длинной: 19.8% против 61.7%. Окно $k = 6$ возвращает успешность до 58.0% (табл. 7). Отношение стоимости к контролю составляет 0.66 против 1.30 у состояния без истории. Стоимость на наблюдаемый успех равна соответственно \$0.1170 и \$0.4867; у контроля — \$0.1899.

Table 8: Те же когорты: медианы контекста и токенов. Cache hit — медиана доли $K/(U + K)$ по выполнениям; столбцы токенов — отдельные медианы.

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Short: control median $T \leq 8$					
Full history	5.6	6.1	9.9	16.4	71.1
Model-written state	2.8	5.4	5.2	16.5	54.7
State + history ($k = 2$)	4.1	9.8	6.9	16.4	40.5
State + history ($k = 6$)	8.8	21.1	7.1	16.4	20.5
Medium: control median $8 < T \leq 20$					
Full history	10.7	20.9	44.4	6.7	79.6
Model-written state	4.0	31.3	58.0	39.4	52.2
State + history ($k = 2$)	6.1	28.7	18.2	16.7	39.7
State + history ($k = 6$)	9.4	38.6	12.9	8.3	25.1
Long: control median $T > 20$					
Full history	48.2	87.7	790.0	30.4	91.2
Model-written state	4.8	131.4	139.6	89.9	43.9
State + history ($k = 2$)	9.1	125.3	65.9	56.1	34.3
State + history ($k = 6$)	17.2	153.0	29.2	36.7	16.5

На длинных траекториях усиливается и преимущество полной истории по использованию кеша. Медианная доля кешированных входных токенов возрастает с 71.1% в короткой когорте до 91.2% в длинной. У состояния без истории и при $k = 6$ в длинной когорте она составляет 43.9% и 16.5%. Сам объём кешированных токенов этого различия не показывает: доля зависит и от некешированного входа.

Когорты показывают, где окно истории полезно, но не задают универсального порога переключения памяти. У задач короткой когорты встречаются отдельные длинные повторы; поэтому эффективность на всей нагрузке и медианная относительная стоимость могут давать разное ранжирование. Траектории зависят от политики агента. Без контролируемого изменения горизонта задачи наблюдаемый результат не устанавливает причинной точки перехода.

9 Можно ли устранить отставание схемой или протоколом

Полезность недавней истории ещё не означает, что исходные взаимодействия принципиально необходимы. Альтернативное объяснение состоит в неудачном способе записи состояния. Отдельный инструмент обновления отличается от совместного контракта SKILL.state, а общая схема может плохо представлять работу в терминале. Для проверки этих объяснений внутри гибридной системы сравниваются сочетания tool/inline-протоколов и исходной/адаптированной схем (табл. 9).

Адаптированная схема при отдельном инструменте даёт 41.4% против 39.7% у опорного варианта. Наблюдаемое улучшение небольшое, а медианная стоимость возрастает с \$0.0120 до \$0.0137. Для восстановления контрольной успешности этого изменения недостаточно.

Table 9: Протокол и схема внутри гибридного состояния без недавней истории, в тех же двух панелях, что и табл. 1. Гибридные сводки не позволяют восстановить парные отношения стоимости, пиковый промпт и cache hit (–).

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Tool / Existing	39.7	8	0.0120	–	21.36
Inline / Existing	35.1	10	0.0165	–	15.25
Tool / Adapted	41.4	7	0.0137	–	21.11
Inline / Adapted	28.7	10	0.0190	–	9.92

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Tool / Existing	–	14.5	9.3	17.3	–
Inline / Existing	–	10.6	1.9	28.6	–
Tool / Adapted	–	18.9	12.3	21.7	–
Inline / Adapted	–	14.1	2.4	33.1	–

Inline-протокол с исходной схемой даёт 35.1% и медианную стоимость $0.0165. Совместное изменение протокола и схемы даёт 28.7% при стоимости $0.0190. Вместо ожидаемого устранения двух ограничений наблюдается ухудшение. Описательная разность эффектов inline-протокола между схемами составляет -8.0 процентного пункта. Проверка такого взаимодействия с учётом задач требует совместных исходов, которых нет в доступной агрегированной сводке.

У inline-вариантов больше медианный выход. Совместное формирование обновления и действия поэтому не оказалось бесплатной заменой отдельного инструмента. Расход может включать текст обновлений, исправления формата и другие изменения траектории; доступные агрегаты не разделяют эти составляющие.

Это сравнение исключает простое объяснение, согласно которому достаточно применить именно проверенную схему или совместный формат ответа, чтобы восстановить результат полной истории. Оно не исключает более удачных схем и реализаций. Измеренные изменения отличаются от возврата недавней истории: последний повышает успешность внутри обеих систем памяти, тогда как выбранные изменения представления не дают сопоставимого восстановления.

10 Что результаты меняют в устройстве памяти

10.1 Полезная идея и разные причины потерь

SKILL.state отделяет рабочий контекст от транскрипта. Эксперименты с программирующим агентом поддерживают эту архитектурную идею, но показывают, почему небольшое состояние ещё не задаёт полноценную политику памяти. Измерены три существенных эффекта. Бедные автоматические записи не удерживают нужную информацию о выполнении: сохранение команд и изменений лучше списка файлов. Удаление недавних взаимодействий увеличивает повторяемость вызовов в пилоте и снижает успешность в больших сравнениях. Наконец, изменение состояния и истории меняет использование кеша, поэтому сокращение логических токенов не является сравнением цен.

Способы компенсации решают разные части проблемы. Автоматические записи избавляют модель от необходимости заново формулировать каждый наблюдаемый факт. Недавнее окно сохраняет детали, которые схема не отобрала. Но их сочетание тоже требует проверки: обогащение

автоматической памяти не давало монотонного улучшения, а цена восстановления истории различалась между реализациями. Результаты поддерживают разделение обязанностей памяти, но не утверждение, что испытанный hybrid превосходит полную историю.

Полная последовательность окон превращает $k = 6$ из необъяснённой константы в измеренный порог: успешность и повторные вызовы перестают улучшаться именно там, но на потолке и полу пилота из 12 задач. Более широкие окна продолжают увеличивать промпт и не убирают постоянную стоимость переписывания состояния на каждом ходе. Часть потерянной информации относится к локальной последовательности выполнения; старые сведения могут оказаться важны на задачах с запасом сложности, а задача с поздней значимостью наблюдения может потребовать записи за пределами любого конечного окна.

10.2 Почему экономия на фиксированном горизонте не закрывает вопрос

В синтетических сравнениях оригинала задан поток событий, тогда как наш агент выбирает действия и момент остановки [Badhe et al., 2026]. Ограничение контекста на фиксированном горизонте не ограничивает дополнительную работу, вызванную политикой памяти. В оригинале есть и публичные интерактивные бенчмарки, поэтому это различие нельзя объявить объяснением всех расхождений. Альтернативными объяснениями остаются модели, схемы и реализации. Аналогично, опубликованное сокращение суммарных токенов не определяет денежную экономию без разбивки входа, выхода и кеша. Сценарий из дополнения S6 показывает эту неопределённость, не претендуя на восстановление счетов авторов.

Когорты не дают универсального правила перехода с транскрипта на состояние. Состояние без истории уступает по успешности во всех когортах, а ограниченное окно восстанавливает значительную часть разрыва и в длинной. Поэтому адаптивной политике следует учитывать потребность в информации и стоимость всей задачи, а не предполагать, что с возрастом траектории недавние взаимодействия становятся не нужны. Следующие различающие проверки — извлечение старых наблюдений из архива и контроль с таким же окном без состояния. Первая проверит восстановление давно удалённых сведений, вторая — дополнительную пользу структурированного блока памяти.

Границы выводов. Результаты относятся к конкретным реализациям на одной модели и наборе задач. Их обобщение ограничивают исследовательский отбор конфигураций, адаптация схемы на тех же задачах, проверки без поправок на повторные задачи и множественность, неполные исходники исполнения и агрегированная отчётность итогового hybrid. Пилот позволяет описать ресурсы и повторяемость, но не проверить эквивалентность успешности. Причинная роль забывания, эффективность адаптивного переключения и извлечения из архива не установлены.

11 Заключение

Структурированное состояние полезно для ограничения активного контекста, но замена истории меняет и возможности агента, и объём работы. Измерения показывают частичные способы компенсации: автоматические записи выполнения богаче списка файлов улучшают поисковый hybrid, а недавние взаимодействия возвращают значительную часть потерянной успешности. Учёт кеша и разбиение по длине контрольных траекторий объясняют, почему ни размер промпта, ни одна общая цена не описывают этот компромисс.

Полное сравнение окон помещает порог между двумя и шестью недавними сообщениями. После шести успешность и повторные вызовы упираются в разрешающую способность пилота, тогда как промпт растёт, а стоимость переписывания состояния на каждом ходе сохраняется; пилот не различает $k = 6$, 12 и ∞. Практический принцип состоит в разделении автоматически сохранённых свидетельств, интерпретации модели и доступа к исходным взаимодействиям. Сжатие ценно,

когда такое сочетание сохраняет способность завершать задачи; адаптивному переключению и извлечению из архива ещё предстоит подтвердить эту пользу.

Список литературы

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. URL `https://arxiv.org/abs/2608.26263`. v3, 02 Sep 2026.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL `https://arxiv.org/abs/2601.11868`. arXiv:2601.11868. Version 2.0 used in this work.

Дополнительные материалы

S1 Данные, определения показателей и чего не хватает

Артефакт содержит записи отдельных выполнений для состояния модели и для сравнения автоматических записей, окончательные агрегированные сводки гибридного сравнения и итоговые сводки пилота окон. Все таблицы и числовые утверждения формируются из этих источников. Гибридные сводки содержат доли решённых и несовпадающие исходы, достаточные для разностей и точных проверок Мак-Немара, но не траектории, промпты отдельных вызовов или cache hit. Исходные сессии пилота недоступны; исходы по задачам перенесены из журнала исследования.

Условные отношения ресурсов считаются на общих успехах:

$$\text{median}_{(i,r):\, Y_{iar}=Y_{i0r}=1} \frac{X_{iar}}{X_{i0r}}.$$

Нулевой знаменатель оставляет отношение неопределённым. Эффективность использует все включённые исходы:

$$\text{Solved/\$} = \frac{\sum Y}{\sum D}, \qquad \text{Solved/Mtok} = \frac{10^6 \sum Y}{\sum (U + K + O)}.$$

Solved/Mtok считает логические токены, а не равную работу для кешированного и некешированного входа. Гибридные показатели эффективности приближённые, поскольку суммы затрат округлены.

S2 Сопоставленные проверки исходов

Table S1: Разности долей решённых задач в процентных пунктах с точными проверками Мак-Немара. Бутстрэп-интервалы доступны только для состояния модели.

Approach	Δ (pp)	95% CI (pp)	p
Model-written memory vs. full history			
Model-written state	-26.4	[-34.1, -18.8]	2.1×10^{-13}
State + history ($k = 2$)	-16.1	[-23.4, -8.8]	9.7×10^{-7}
State + history ($k = 6$)	-2.3	[-7.7, +3.4]	0.488
Hybrid memory vs. full history			
Hybrid, no failure records	-13.2	–	4.3×10^{-4}
Hybrid state	-12.1	–	0.001
Hybrid + adapted schema	-10.3	–	0.008
Hybrid + inline	-16.7	–	5.7×10^{-5}
Hybrid + inline + adapted schema	-23.0	–	2.3×10^{-8}
Hybrid + history ($k = 2$)	-10.9	–	0.001
Hybrid + history ($k = 6$)	-3.4	–	0.392
Hybrid variants vs. hybrid state			
Hybrid, no failure records	-1.1	–	0.824
Hybrid + adapted schema	+1.7	–	0.678
Hybrid + inline	-4.6	–	0.185
Hybrid + inline + adapted schema	-10.9	–	0.001
Hybrid + history ($k = 2$)	+1.1	–	0.824
Hybrid + history ($k = 6$)	+8.6	–	0.011

Table S2: Разности долей решённых задач относительно полной истории внутри когорт горизонта, заданных контролем (состояние модели).

Approach	Δ (pp)	p
Short: control median $T \leq 8$		
Model-written state	-18.3	2.2×10^{-4}
State + history ($k = 2$)	-10.8	0.031
State + history ($k = 6$)	-1.1	1.000
Medium: control median $8 < T \leq 20$		
Model-written state	-20.7	0.003
State + history ($k = 2$)	-11.5	0.076
State + history ($k = 6$)	-2.3	0.815
Long: control median $T > 20$		
Model-written state	-42.0	1.9×10^{-8}
State + history ($k = 2$)	-27.2	5.9×10^{-5}
State + history ($k = 6$)	-3.7	0.648

Table S3: Стоимость и эффективность с учётом неудач. Медиана и среднее стоимости отвечают на иные вопросы, чем суммарная стоимость на успех; ни одна величина не оценивает стоимость повторов до успеха.

Approach	Median cost ($)	Mean cost ($)	Cost/success ($)	Solved/$	Solved/Mtok
Model-written memory					
Full history	0.0137	0.0684	0.1116	8.96	0.421
Model-written state	0.0239	0.0568	0.1630	6.14	1.245
State + history ($k = 2$)	0.0151	0.0493	0.1091	9.16	1.964
State + history ($k = 6$)	0.0154	0.0453	0.0768	13.02	2.718
Hybrid memory (efficiencies approximate)					
Full history	0.0104	0.0245	0.0473	21.13	–
Hybrid state	0.0120	0.0186	0.0468	21.36	–
Hybrid + adapted schema	0.0137	0.0196	0.0474	21.11	–
Hybrid + inline	0.0165	0.0230	0.0656	15.25	–
Hybrid + inline + adapted schema	0.0190	0.0290	0.1008	9.92	–
Hybrid + history ($k = 2$)	0.0122	0.0189	0.0462	21.65	–
Hybrid + history ($k = 6$)	0.0144	0.0242	0.0501	19.95	–

S3 Стоимость по всем сохранённым выполнениям

S4 Чувствительность стоимости автоматических записей к подмножеству успехов

Отношения на переменном подмножестве используют общие успехи каждой конфигурации с полной историей; на фиксированном — задачи, решённые всеми конфигурациями. Богатейшие варианты стоят 0.92 и 0.94 от полной истории в первом случае, 1.03 и 1.17 во втором: экономия относительно ссылок на файлы сохраняется, относительно полной истории — нет.

Table S4: Чувствительность стоимости автоматических записей. Значения — медианы отношений к полной истории внутри задачи.

Runtime-owned information	Variable subset Cost	Fixed subset Cost	Fixed subset Turns	Fixed subset Output
File references	2.31×	2.50×	1.61×	4.03×
Files, changes, commands	1.20×	1.07×	0.77×	1.56×
+ diagnostics, environment, tests	0.92×	1.03×	0.75×	1.75×
+ failed-attempt records	0.94×	1.17×	0.79×	1.73×

S5 Отношения пилота окон по повторам

Table S5: Пилот окон: медианные отношения стоимости и пикового промпта к полной истории в том же повторе, отдельно для каждого повтора.

Approach	Cost ratio Repeat 1	Cost ratio Repeat 2	Cost ratio Repeat 3	Peak ratio Repeat 1	Peak ratio Repeat 2	Peak ratio Repeat 3
Model-written state	2.35	2.09	1.94	0.36	0.38	0.29
State + history ($k = 2$)	1.84	1.57	1.13	0.63	0.57	0.50
State + history ($k = 6$)	1.82	1.67	1.68	0.90	0.79	0.85
State + history ($k = 12$)	2.25	1.58	1.63	1.04	1.31	1.04
State + history ($k = \infty$)	1.96	2.00	1.55	1.40	1.69	1.31

S6 Сценарий кеш-скидки для исходных результатов

Исходная публикация приводит только суммы токенов. Умножение её отношения токенов полная история/состояние на предполагаемую кеш-скидку (hit полной истории 0.93, состояния 0.72, цена кеша 0.198 от входа) даёт приведённый ниже показатель. Он считает все токены кешируемым входом и служит иллюстрацией чувствительности, а не реконструкцией счёта.

Table S6: Опубликованное отношение токенов полная история/состояние и показатель с учётом скидки по горизонту событий. Значения больше единицы благоприятны для состояния.

Event horizon (T)	Published token ratio (Full / State)	Discounted-token proxy (Full / State)
10	1.61×	0.97×
25	2.90×	1.74×
50	5.69×	3.42×
100	19.04×	11.45×
200	21.32×	12.82×